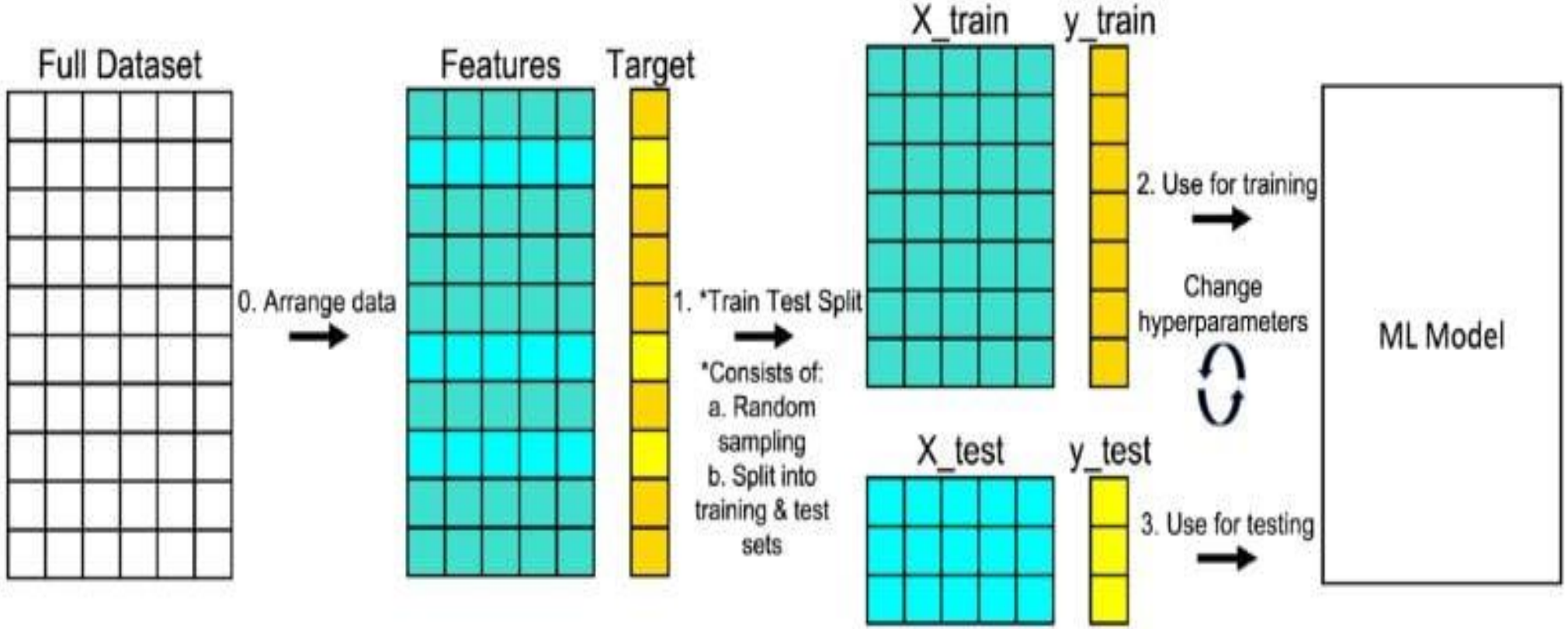
Lab 5 – Stepwise Regression

Guided Steps with Examples

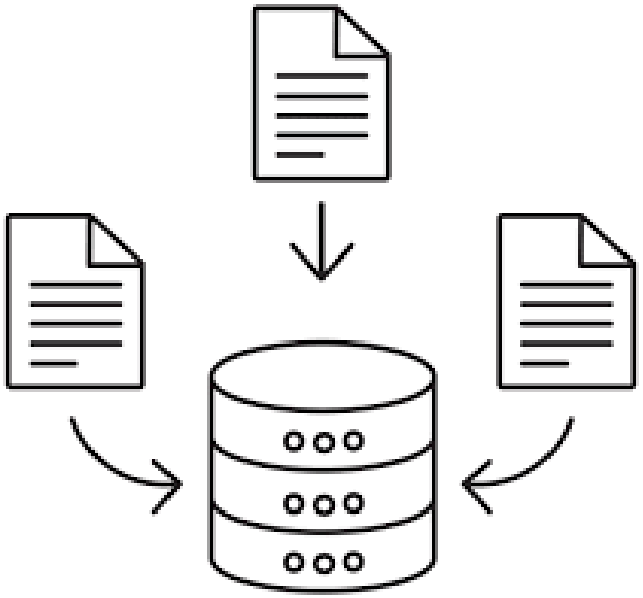
Objective


Full Dataset
0. Arrange data
Features
Target
1. *Train Test Split
*Consists of:
a. Random sampling
b. Split into training & test sets
X_train
y_train
X_test
y_test
2. Use for training
Change hyperparameters
3. Use for testing
ML Model

Steps

1. Collect the data
2. Visualize the data
3. Split data (train/test)
4. Build the regression model
5. Evaluate the model
6. Predict new values

Step 1: Collect the data





```
# Import the dataset as csv file
import pandas as pd
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('Cellphone.csv')
```

Step 2: Reading the data and start preprocessing (Lab 4)

- In this section we will do the data preprocessing and cleaning. Usually, this is the most important part in any Machine Learning project. Clean data means high quality data which will result in a very good performance of the ML model.

2a. Checking size

```
# here we will read the data, check the data shape, and view the first 5 rows

print("Dataset shape is", df.shape)
print()
df.head()
```

```
Dataset shape is (161, 15)
```

	project_id	Product_id	Price	Sale	weight	resoloution	ppi	cpu core	cpu freq	internal mem	ram	RearCam	Front_Cam	battery	thickness
0	1	203	2357	10.0	135.0	5.2	424	8	1.35	16.0	3.000	13.00	8.0	2610	7.4
1	2	880	1749	10.0	125.0	4.0	233	2	1.30	4.0	1.000	3.15	0.0	1700	9.9
2	3	40	1916	10.0	110.0	4.7	312	4	1.20	8.0	1.500	13.00	5.0	2000	7.6
3	4	99	1315	11.0	118.5	4.0	233	2	1.30	4.0	0.512	3.15	0.0	1400	11.0
4	5	880	1749	11.0	125.0	4.0	233	2	1.30	4.0	1.000	3.15	0.0	1700	9.9

2b. Summary

```
summary = df.describe() # describe the dataset + return the description & save it in summary
summary.to_csv('summary_cellphone.csv') # save the summary in an csv file named 'summary_cellphone.csv'

summary
```

	project_id	Product_id	Price	Sale	weight	resoloution	ppi	cpu core	cpu freq	internal mem	ram	RearCam	Front_Cam
count	161.000000	161.000000	161.000000	160.000000	160.000000	161.000000	161.000000	161.000000	160.000000	161.000000	161.000000	160.000000	161.000000
mean	81.000000	675.559006	2215.596273	625.100000	170.585000	5.209938	335.055901	4.857143	1.498788	24.501714	2.204994	10.361875	4.503106
std	46.620811	410.851583	768.187171	1550.784744	93.158301	1.509953	134.826659	2.444016	0.599459	28.804773	1.609831	6.197485	4.342053
min	1.000000	10.000000	614.000000	10.000000	66.000000	1.400000	121.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	41.000000	237.000000	1734.000000	36.750000	134.075000	4.800000	233.000000	4.000000	1.200000	8.000000	1.000000	5.000000	0.000000
50%	81.000000	774.000000	2258.000000	106.000000	153.500000	5.150000	294.000000	4.000000	1.400000	16.000000	2.000000	12.000000	5.000000
75%	121.000000	1026.000000	2744.000000	382.750000	170.250000	5.500000	428.000000	8.000000	1.875000	32.000000	3.000000	16.000000	8.000000
max	161.000000	1339.000000	4361.000000	9807.000000	753.000000	12.200000	806.000000	8.000000	2.700000	128.000000	6.000000	23.000000	20.000000

2c. Drop Non-Necessary Columns

```
df.drop(columns = ['project_id', 'Product_id'], inplace = True) # drop columns specified
                                                                # inplace = True means modify the original df

df.head()
```

	Price	Sale	weight	resoloution	ppi	cpu core	cpu freq	internal mem	ram	RearCam	Front_Cam	battery	thickness
0	2357	10.0	135.0	5.2	424	8	1.35	16.0	3.000	13.00	8.0	2610	7.4
1	1749	10.0	125.0	4.0	233	2	1.30	4.0	1.000	3.15	0.0	1700	9.9
2	1916	10.0	110.0	4.7	312	4	1.20	8.0	1.500	13.00	5.0	2000	7.6
3	1315	11.0	118.5	4.0	233	2	1.30	4.0	0.512	3.15	0.0	1400	11.0
4	1749	11.0	125.0	4.0	233	2	1.30	4.0	1.000	3.15	0.0	1700	9.9

2d. Checking for Missing Values

- Now we usually have 2 solutions for this problem:

1. Remove the whole row that contains missing values.

2. Replace missing values with the mean, median or mode (most frequent value) of that column.

Check for missing values

```
[ ]  print(df.isnull().sum()) # check for missing values in each column
```

```
Price           0
Sale            1
weight          1
resoloution     0
ppi             0
cpu core        0
cpu freq        1
internal mem    0
ram             0
RearCam         1
Front_Cam       0
battery         0
thickness       0
dtype: int64
```

```python
df1 = df.dropna(inplace = False)  # drop missing values
                                  # inplace = True, modify the dataset
                                  # inplace = False, do NOT modify the dataset and return a modified copy
print(df1.isnull().sum())
print()
print("Dataset shape before dropping rows with missing values:", df.shape)
print()
print("Dataset shape after dropping rows with missing values:", df1.shape) # the shape will change as entire rows are dropped
```

```
Price          0
Sale           0
weight         0
resoloution    0
ppi            0
cpu core       0
cpu freq       0
internal mem   0
ram            0
RearCam        0
Front_Cam      0
battery        0
thickness      0
dtype: int64

Dataset shape before dropping rows with missing values: (161, 13)

Dataset shape after dropping rows with missing values: (157, 13)
```

Filling missing values with median

```
Price          0
Sale           1
weight         1
resoloution    0
ppi            0
cpu core       0
cpu freq       1
internal mem   0
ram            0
RearCam        1
Front_Cam      0
battery        0
thickness      0
dtype: int64
```

Missing values before replacing

```python
# Dataset before replacing
print("Dataset checking missing values before filling rows with median:\n\n", df.isnull().sum())
print()
print("Dataset median before filling:\n\n", df.median(numeric_only=True))
print()

# fill the missing values with the median of the column

df['Sale'] = df['Sale'].fillna(df['Sale'].median())
df['weight'] = df['weight'].fillna(df['weight'].median())
df['cpu freq'] = df['cpu freq'].fillna(df['cpu freq'].median())
df['RearCam'] = df['RearCam'].fillna(df['RearCam'].median())

# or we can use a for loop instead of writing the same line again
col_with_missing = ['Sale', 'weight', 'cpu freq', 'RearCam']

for col in col_with_missing:
  df[col] = df[col].fillna( df[col].median() )

# Dataset after replacing
print("Checking missing values after filling rows with median:\n\n", df.isnull().sum())
print()
print("Dataset median after filling:\n\n", df.median(numeric_only=True), "\n")
```

Fillna method

```
Price          0
Sale           0
weight         0
resoloution    0
ppi            0
cpu core       0
cpu freq       0
internal mem   0
ram            0
RearCam        0
Front_Cam      0
battery        0
thickness      0
dtype: int64
```

Missing values after replacing

Step 3: Split data (train/test)

- Import necessary libraries

```
!pip install stepwise_regression
```

```
Collecting stepwise_regression
  Downloading stepwise_regression-1.0.3-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.11/dist-packages (from stepwise_regression) (0.14.4)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from stepwise_regression) (2.2.2)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages (from pandas->stepwise_regression) (2.
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->stepwise_regres
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->stepwise_regression) (202
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->stepwise_regression) (2
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /usr/local/lib/python3.11/dist-packages (from statsmodels->stepwise_regre
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-packages (from statsmodels->stepwise_regression)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.11/dist-packages (from statsmodels->stepwise_regressi
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->stepw
Downloading stepwise_regression-1.0.3-py3-none-any.whl (3.3 kB)
Installing collected packages: stepwise_regression
Successfully installed stepwise_regression-1.0.3
```

```
from stepwise_regression import step_reg
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Step 3: Split data (train/test)

- Split data to inputs and outputs.

```
# 1st way - using the loc method

print("\n**************************Using the LOC method**************************\n")

x = df.loc[:, df.columns != "Price"]   # get all rows + all cols except "Price"
y = df.loc[:, df.columns == "Price"]  # get all rows + "Price" col

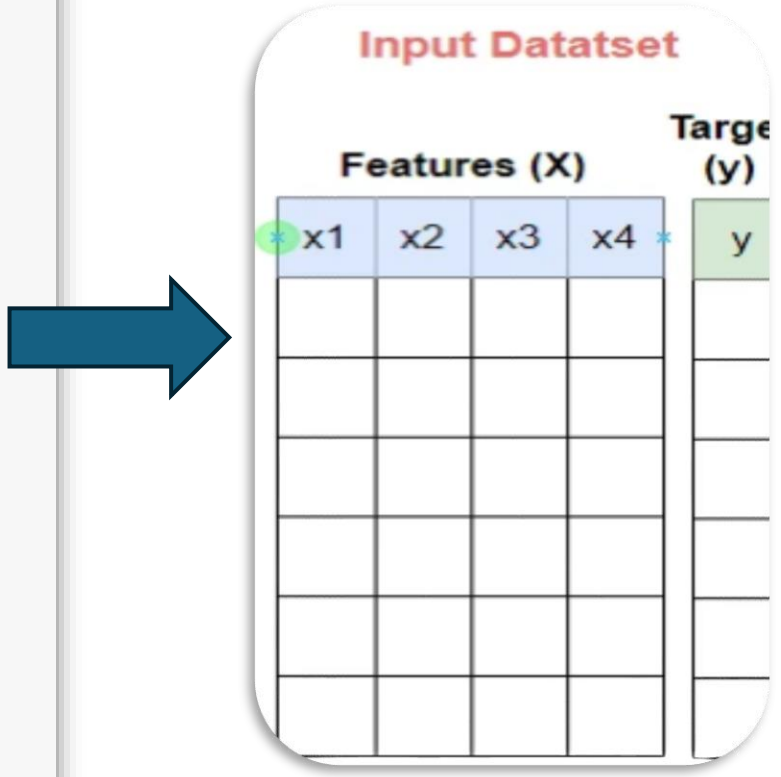
display(x)
print()
display(y)
print()

# 2nd way - using the iloc method

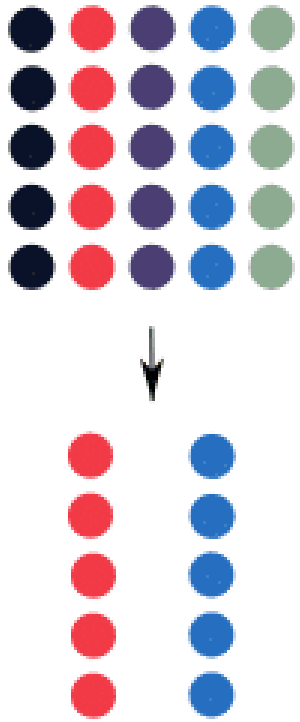
print("\n**************************Using the iLOC method**************************\n")

x = df.iloc[:, 1:13]  # get all rows + columns from 1 to 12 (col 0 not included)
y = df.iloc[:, 0]     # get all rows + 1st col (col 0)

display(x)
print()
display(y)
print()
```



Feature Stepwise Selection



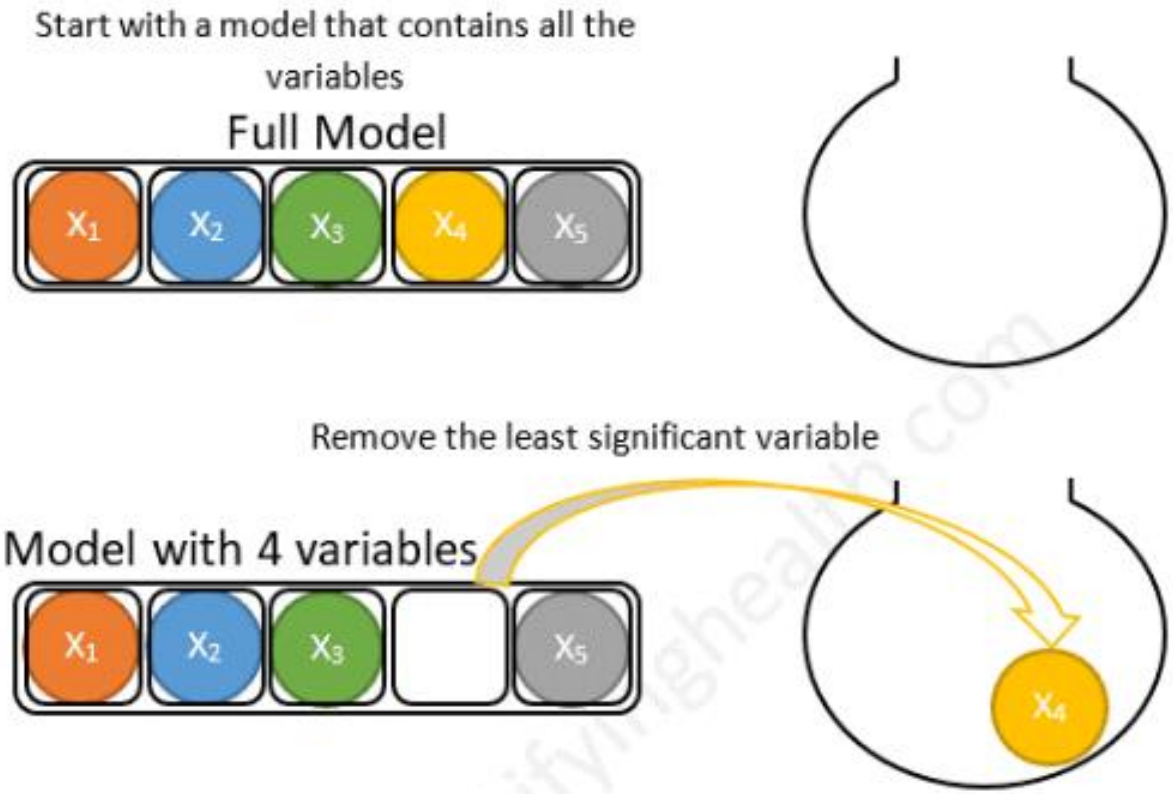
Sample size = number of variables

Stepwise selection

Sample size >> number of variables

More Generalizable Model

Backward stepwise selection example with 5 variables:



Stepwise Regression

- This code will view the most important features based on our **P value which is 0.05**. So, all features selected are having **P value less than 0.05** which means selected_features are significant at 95% Critical Intervals (alpha=0.05).

```
Drop RearCam                                with p-value 0.529604
Drop Front_Cam                              with p-value 0.093113
Drop Sale                                   with p-value 0.13333
================================================================
['weight',
 'resoloution',
 'ppi',
 'cpu core',
 'cpu freq',
 'internal mem',
 'ram',
 'battery']
```

```
# Define the model and fit it

p_value = 0.05

selected_features = step_reg.backward_regression(x, y, p_value, verbose=True)
print("="*80)
selected_features
```

```
x_modified = df[selected_features]
x_modified
```

	weight	resoloution	ppi	cpu core	cpu freq	internal mem	ram	battery
0	135.0	5.20	424	8	1.350	16.0	3.000	2610
1	125.0	4.00	233	2	1.300	4.0	1.000	1700
2	110.0	4.70	312	4	1.200	8.0	1.500	2000
3	118.5	4.00	233	2	1.300	4.0	0.512	1400
4	125.0	4.00	233	2	1.300	4.0	1.000	1700
...	...	...	...	...	...	...	...	...
156	178.0	5.46	538	4	1.875	128.0	6.000	4080
157	170.0	5.50	534	4	1.975	128.0	6.000	3400
158	150.0	5.50	401	8	2.200	64.0	4.000	3000

Splitting the data - Holdout method 75% training and 25% testing

```
# train_test_split function
# it takes a lot of input parameters but we will use only 4 and they are:
  # 1. x → input
  # 2. y → output
  # 3. test_size = 0.25 → this means we want 75% training and 25% testing (25% = 0.2)
  # 4. random_state = 2 → this could be any number, the main objective of it is when
  #    you run the program again it will do the same random split

# it retruns 4 variables - x_train, x_test, y_train, y_test

x_train, x_test, y_train, y_test = train_test_split(x_modified, y, test_size = 0.25, random_state = 2)
```

Step 4: Build the regression model

Model building and training

```
[ ]  # now we will create the new model and will fit it (train) on our data

     model = LinearRegression()
     model.fit(x_train, y_train)
```

```
LinearRegression
LinearRegression()
```

Step 5: Evaluate the model (Test)

Testing

```
# here we will use predict function and the testing data (x_test) to test the
# model and store its predictions in a variable named "predictions"

predictions = model.predict(x_test)
```

Evaluation

```
# here we will use the predictions and the original results (y_test) to evaluate the model
# we will use these metrices:
# Mean Square Error (MSE)
# Mean Absolute Error (MAE)
# R2 Squared (R2)

print("Mean Squared Error:", mean_squared_error(y_test, predictions))
print()
print("Mean Absolute Error:", mean_absolute_error(y_test, predictions))
print()
print("R2 Squared:", r2_score(y_test, predictions))
```

```
Mean Squared Error: 27194.57501536442

Mean Absolute Error: 141.89464081639088

R2 Squared: 0.9439060392439897
```

Conclusion



- Regression is about finding the relationship
- It helps in making predictions
- It is used to **select the best variables** (features) for a regression model — whether it's **linear** or **multiple linear**.
- Stepwise regression is a **feature selection method**.
- It is used in **linear and multiple linear regression**.
- Not usually applied in non-linear regression.
- Regression only works with **numerical values**